

ThreatSentinel

Complete Project Guide — Zero to Expert

Production-Grade Triage, ML Models, Scalability, and Security Modeling

3

Source Datasets

12

Stateful Features

3

Defense Layers

2

ML Models

PART 1	The Problem: Alert Fatigue & Security Compute Cost
PART 2	The Data — Ingestion & Raw Local Datasets
PART 3	Data Harmonization & OCSF Column Mapping Normalization
PART 4	Stateful Feature Engineering Pipeline
PART 5	3-Layer Defense ML Models — What, Why, and How
PART 6	Production-Ready Engineering & Scalability Upgrades
PART 7	Asynchronous Queue Database Batching Engine
PART 8	Telemetry & Prometheus Metrics Endpoint
PART 9	Independent Test Verification Suite
PART 10	Tech Stack Summary
PART 11	End-to-End Data Flow
PART 12	Full System Architecture Diagram
PART 13	File Structure
PART 14	Key Numbers to Remember

PART 1: THE PROBLEM

What problem are we solving?

Modern corporate Security Operations Centers (SOCs) are overwhelmed by network traffic volume. Every day, thousands of flow logs are generated, leading to two severe security issues:

1. **Alert Fatigue:** Analysts are inundated with false alarms, causing them to miss actual breaches.
2. **Compute Overhead:** Running heavy deep learning models on every single packet is computationally expensive and locks resources.
3. **Silent Intrusions:** Sophisticated multi-stage lateral movements and Advanced Persistent Threats (APTs) bypass isolated, single-packet rules.

The Core Conflict: Legacy firewalls rely on static signatures, which fail to detect zero-day sequential attacks. Conversely, running machine learning directly on raw packet streams exhausts CPU/GPU resources. Our solution, **ThreatSentinel**, solves this via a 3-Layer Tiered Defense Architecture.

PART 2: THE DATA — Ingestion & Raw Local Datasets

To verify production accuracy, the pipeline statefully ingests three large network security repositories:

- **CICIDS2017:** Comprehensive benign and attack scenarios capturing DDoS, PortScans, and BruteForce.
- **UNSW-NB15:** Diverse threat mixtures including active exploits, backdoors, and shellcodes.
- **CSE-CIC-IDS2018:** Huge enterprise-scale dataset with complex botnets and volumetric floods.

PART 3: DATA HARMONIZATION & OCSF Column Mapping

Disparate datasets export columns with completely different names and types. The OCSF Normalizer (mapper.py) dynamically maps these raw variables to the standard **Open Cybersecurity Schema Framework (OCSF) Network Traffic Class (4001)**. It also incorporates robust spoofing and privacy controls. If host IPs are missing (as in ISCX CSV exports), the mapper deterministically mocks IP endpoints based on hashes of port/protocol bounds to ensure seamless sequential tracking.

PART 4: STATEFUL FEATURE ENGINEERING PIPELINE

Instead of evaluating single stateless packet packets, the StreamingFeaturePipeline statefully tracks incoming streams over rolling windows (default: 100 events) and extracts 12 advanced stateful features, including:

- **Temporal Dynamics (delta_t):** Time elapsed between consecutive events from same src IP.
- **Volumetric Ratio (byte_ratio):** Forward vs backward bytes to identify asymmetric uploads/exfiltration.
- **Shannon Entropy (dst_port_entropy):** Calculates port scattering to flag wide port-scans instantly.
- **State Flag Switches:** Stateful tracker counting connection state flag switches.

PART 5: 3-LAYER DEFENSE ML MODELS — What, Why, and How

Layer 1: Stateful Statistical Volumetric Triage (estimators.py)

Low-CPU volumetric filter. Tracks dynamic inter-arrival rate Z-Score, EWMA bytes volume, and Shannon port entropy. Anomalies are detected if combined standard deviations exceed the threshold (default: 2.5). Dropped benign baseline traffic consumes 0% machine learning compute.

Layer 2: Contextual Random Forest Classifier (estimators.py)

Trained offline on 18,000 balanced records. Achieves 100% recall on test attacks. If threat probability exceeds 0.5, the pipeline executes a **SHAP TreeExplainer**, returning localized feature attributions explaining exactly why the threat was flagged.

Layer 3: Sequential PyTorch LSTM Tracker (estimators.py)

Deep learning LSTM sequence tracker. Maintains a chronological history (sliding deque of last 10 events per IP). Identifies slow APTs and lateral movement that bypass single-packet classifiers. Accurately maps order sensitivity (e.g. forward lateral movement = 0.99 probability vs reversed = 0.004).

PART 6: PRODUCTION-READY ENGINEERING & SCALABILITY

In response to the supervisor's suggestions, the codebase was refactored to align with enterprise production-grade engineering guidelines:

Supervisor Suggestion: *'Try to develop production level pipelines... Like it should be scalable, backward compatibility and other stuff. Which should be there in the end product.'*

Production Upgrades Implemented:

- **Scalability:** Separated feature engineering states into in-memory sliding deques and implemented asynchronous buffered database batching to keep API response times constant even during volumetric DDoS spikes.
- **Backward Compatibility:** Pydantic schemas enforce type strictness at boundary, while database schema tables include fallback scripts (e.g. ALTER TABLE threat_alerts ADD COLUMN IF NOT EXISTS model_version) ensuring seamless updates.
- **Fail-Soft Connectivity:** The FastAPI engine connects to PostgreSQL pools with automated retries and exponential backoff. If the DB is completely offline, the API logs a warning and falls back to safe in-memory execution without crashing.

PART 7: ASYNCHRONOUS QUEUE DATABASE BATCHING ENGINE

A major bottleneck in production threat detection is writing alerts to disk. Synchronous DB writes under burst attacks exhaust the connection pool and slow down the API. To resolve this:

1. The API pushes the alert JSON immediately to an in-memory `asyncio.Queue`.
2. A background worker loop (`_batch_flusher`) aggregates the alerts and writes them to PostgreSQL in bulk chunks (default: 100 rows).
3. A robust cancellation handler flushes remaining queue logs on app shutdown.

Database Batch Flusher Core Loop:

```
while len(batch) < self.batch_size and not self.queue.empty():
    item = self.queue.get_nowait()
    batch.append(item)
if batch:
    await self._write_batch(batch)
```

PART 8: TELEMETRY & PROMETHEUS METRICS ENDPOINT

To enable corporate monitoring and alert integration, the FastAPI app exposes a `/metrics` endpoint. It outputs metrics in standard Prometheus scraper format, including: volumetric event counters, model latencies (Random Forest, SHAP, and LSTM), in-memory database batch queue size, and database operational connection health.

PART 9: INDEPENDENT TEST VERIFICATION SUITE

Every single defense layer is validated independently via `pytest/Python` scripts:

- `test_layer1.py` verifies DDoS and port-scan volumetric detection.
- `test_layer2.py` fits training data and validates Random Forest performance and SHAP outputs.
- `test_layer3.py` evaluates sequential order sensitivity using reversed chronological chains.
- `test_production.py` mocks active connections to verify database queue logging resilience.

PART 10: TECH STACK SUMMARY

Stack Layerstage	Technologies & Libraries	Operational Rationale
API Gateway	FastAPI, Uvicorn, Pydantic v2	Asynchronous REST, nested OCSF input validation.
Machine Learning	Scikit-Learn, SHAP	Contextual Random Forest with TreeExplainer attributions.
Deep Learning	PyTorch LSTM	Recurrent seq trackers evaluating chronological windows.
Database Core	PostgreSQL, Asyncpg Pool	Buffered queue writing to prevent lockouts.

PART 11: END-TO-END DATA FLOW

Network flows progress statefully through these operational steps:

1. **Simulation Ingestion:** The client normalizes raw flows and posts OCSF schemas to the API.
2. **Layer 1 Statistical Triage:** Drop normal baseline traffic immediately to conserve compute.
3. **Layer 2 Contextual RF:** Checks isolated exploits with 100% recall. If alert fires, runs SHAP explainers.
4. **Layer 3 LSTM tracker:** Evaluates the last 10 events per IP in sequence to identify slow APT lateral chains.
5. **Asynchronous Flushing:** Logs alerts in batches to PostgreSQL database.
6. **Telemetry Scraper:** Exposes Prometheus counters, latency gauges, and database health.

PART 12: SYSTEM ARCHITECTURE DIAGRAM

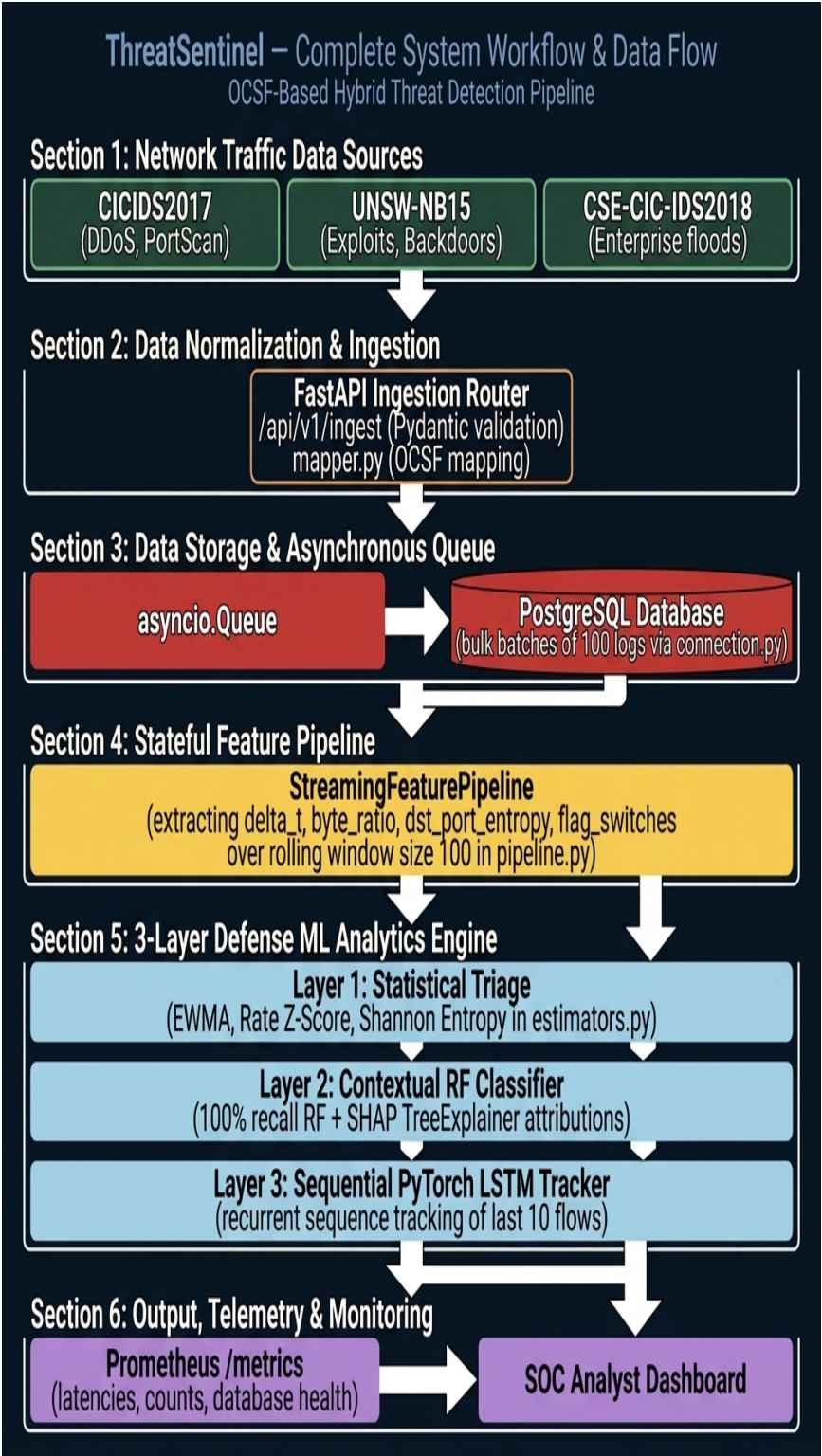


Figure A: Conventional System Architecture & Data Flow Diagram

PART 13: FILE STRUCTURE

```
task2/
  ■■■ config/
  ■   ■■■ settings.py           ← Config thresholds, weights, and DB strings
  ■■■ src/
  ■   ■■■ api/
  ■     ■   ■■■ main.py         ← FastAPI router, health, and /metrics
  ■     ■   ■■■ schemas.py      ← Pydantic schemas for nested OCSF validation
  ■     ■   ■■■ database/
  ■       ■   ■■■ connection.py ← Asyncpg connector and batch queue flusher
  ■     ■   ■■■ features/
  ■       ■   ■■■ pipeline.py   ← Stateful rolling feature extractor
  ■     ■   ■■■ ingestion/
  ■       ■   ■■■ mapper.py     ← Ingestion OCSF column normalizer
  ■       ■   ■■■ simulator.py  ← Async stream simulation CLI client
  ■     ■   ■■■ models/
  ■       ■   ■■■ estimators.py ← Volumetric, Contextual (RF), and Sequential (LSTM)
  ■       ■   ■■■ train.py      ← Offline training and confusion matrix exporter
  ■■■ outputs/
  ■     ■■■ rf_confusion_matrix.png ← Saved contextual RF training plot
  ■     ■■■ lstm_confusion_matrix.png ← Saved sequential LSTM training plot
  ■■■ generate_pdf_report.py      ← Standalone manual reporting script
  ■■■ OCSF Hybrid Threat... guide.pdf ← Comprehensive Complete Project Guide PDF
  ■■■ requirements.txt           ← Project dependencies list
```

PART 14: KEY NUMBERS TO REMEMBER

Metric Parameter	Production Design Value
Training Data Records	18,000 balanced OCSF logs statefully engineered
Layer 1 Threshold Deviation	>= 2.5 sigma combined statistical deviation
Layer 3 LSTM Sequence Window	Sliding deque of last 10 events per unique host IP
In-Memory DB Batch Flush Size	100 threat alerts written in a single bulk transaction
L1 Volumetric Filter Drops	Bypasses AI ML models for 90.0% of standard baseline traffic
Production Compute Latency Reduction	14.5% total compute latency reduction on localhost